

Week 5 • Accounts and sharing: authentication, ACLs, roles

Technical Interaction Design · ITU · Autumn 2026

Built 18 September 2026 · commit eef5868

Latest version: <https://itu-tid.github.io/lecture-notes-pdf/Week-05.pdf>

CONTENTS

Authentication and Authorization (in Parse)

Part 1: Authentication

- Parse gives you accounts, login and the current user without your writing any of it

Part 2: Authorization

- Reading

Web Service APIs

- For your projects
- Not discussed in the course

Relationships Between Object Classes

- Todos belong to lists
- The same idea has three names, depending on the context
- One-to-many relationships are done with pointers
- Many-to-many relationships are done with a join table
- Note: do not model relationships with `Parse.Relation`
- Note: do not model relationships with arrays
- The notation matters less than being able to explain your model

Authentication and Authorization (in Parse)

Source: [Lectures/Technical/Backend/Authorization-and-ACL-in-Parse.md](#) — something unclear? [Open an issue](#), or send a pull request.

Two words that sound alike and mean different things, which is why they belong in the same lecture:

- **Authentication** — proving that a user is who they say they are. *Who are you?*
- **Authorization** — deciding what that user is then allowed to do. *What are you allowed to touch?*

The first half of today gives your app accounts. The second half stops those accounts from reading each other's data. Neither half is worth much without the other: accounts that everyone can read through are theatre, and permissions with nobody to attach them to are unusable.

Part 1: Authentication

Parse gives you accounts, login and the current user without your writing any of it

The Javascript Parse SDK helps you manage user accounts and track the logged in user. `_User` is a class that already exists in your database, with `username`, `password` and `email` already on it, and with the password already being hashed for you.

Signing up and logging in

```
import { useState } from 'react';
import Parse from 'parse';

export default function AuthPage({ onAuthenticated }) {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');

  async function handleSignUp(e) {
    e.preventDefault();
    setError('');
    try {
      const user = new Parse.User();
      user.set('username', username);
      user.set('password', password);
      await user.signUp();
      onAuthenticated(user);
    } catch (err) {
      setError(err.message);
    }
  }

  async function handleLogin(e) {
    e.preventDefault();
    setError('');
    try {
      const user = await Parse.User.login(username, password);
      onAuthenticated(user);
    } catch (err) {
      setError(err.message);
    }
  }

  return (
    <div>
      <h1>Welcome</h1>
      {error && <p style={{ color: 'red' }}>{error}</p>}

      <form>
        <input
          type="text"
          placeholder="Username"

```

```

        value={username}
        onChange={(e) => setUsername(e.target.value)}
      />
      <input
        type="password"
        placeholder="Password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />
      <button onClick={handleSignUp}>Sign Up</button>
      <button onClick={handleLogin}>Log In</button>
    </form>
  </div>
);
}

```

What happens:

- `user.signUp()` creates a new user, and logs them in
- `Parse.User.logIn()` logs in an existing one
- Both of them manage the session for you
- Log out with `await Parse.User.logout()`

Showing the login page, without a router

Both handlers finish by calling `onAuthenticated`. They do *not* navigate anywhere, because we have no router yet — that is week 6. We do not need one:

```

function App() {
  const [user, setUser] = useState(Parse.User.current());

  if (!user) {
    return <AuthPage onAuthenticated={setUser} />;
  }

  return <ToDoList user={user} onLogOut={() => Parse.User.logout().then(() =>
    setUser(null))} />;
}

```

This is [conditional rendering](#) again, in its whole-screen form: either the app, or the way in. A router will later give each of these its own URL, which is a real improvement — but notice that what a router adds here is *addressability*, not the gating. The gating is this `if`.

Getting the current user

It would be silly if the user had to login every time they opened the app, so Parse stores the logged-in user in `localStorage` and hands them back to you:

```
const currentUser = Parse.User.current();
if (currentUser) {
  // do stuff with the user
} else {
  // show the signup or login page
}
```

This is why `useState(Parse.User.current())` above is safe as a `useState` initializer while `fetchTodos()` was not: `current()` reads from `localStorage` and is synchronous. No network, no waiting, no third state.

Logging out

```
await Parse.User.logout();
// Parse.User.current() is now null
```

Associating the to-dos with their owner

A to-do now belongs to somebody. That is one more pointer, exactly like the `list` pointer from last week:

```
item.set("owner", Parse.User.current());
```

and then the query that fetches them says so:

```
const query = new Parse.Query(TodoItem);
query.equalTo("owner", Parse.User.current());
const results = await query.find();
```

Why is this critical?

- It allows the query "show me only *my* todos"
- It is what data privacy will be built on
- It is what the access control in Part 2 attaches to

But it is not yet security. That `equalTo` is a convenience for the client, not a rule on the server. A user who opens the developer tools, or who talks to the REST API directly, can simply not send that constraint — and today your server would happily answer. Which is what the rest of the lecture is about.

More advanced features, for later

- [Email Verification](#)
- [Security of User Objects](#) - only a user can modify their own data
- [Resetting Passwords](#)

Something unclear in this section? Open an issue.

Part 2: Authorization

Motivation

Why can you not keep the Parse API keys perfectly secret?

- Remember the architectural diagram from the beginning of the course? `bundle.js` is sent to the browser...
- The JavaScript code of your web application can be inspected by another web programmer.

See it, rather than take my word for it

Open your own to-do app, open the developer tools on the **Network** tab, and tick a checkbox. One request appears. Click it, and look at what your app sent:

```
POST https://parseapi.back4app.com/parse/classes/ToDoItem/AbC123xY

{"done":true, "_method":"PUT",
 "_ApplicationId":"YOUR_APP_ID",
 "_JavaScriptKey":"YOUR_JS_KEY",
 "_SessionToken":"r:aBcD1234..."}

```

There they are. Not hidden in the bundle, not obfuscated — **written in plain text in the body of every single request**, by your own code, because the server has no other way of knowing who is calling.

Anybody who can open your app can read them. That is not a bug in Parse, and there is no setting that fixes it: a key that the browser must send is a key the browser's owner can read.

Now look at the two credentials in there, because the difference between them is this entire lecture:

- `_ApplicationId` and `_JavaScriptKey` say **which application is calling**. Every visitor has them. They identify your app, and they prove nothing about the person using it.
- `_SessionToken` appeared when you logged in an hour ago. It says **who is calling**. It is yours, it is secret, and it is the only thing in this request that the server can use to tell you from anybody else.

Everything we build today hangs off that second line. The keys cannot protect anything, so the protection has to be attached to the user.

And two more things are visible in that same request, both of which matter shortly: the **class name** (`ToDoItem`) and the **object id** — so a stranger now knows what your tables are called, and can address individual rows in them.

What happens if I access your repository and find your AppID and JSKey?

- Read info that is not meant for me

- Delete useful information
- Store my movie collection in your tables
- etc.

That is - **only if you have made your tables public**

- When we created the DB we were asked about access control and we agreed to make everything public because we're working on an MVP.
- Now it's time to harden the security of our database

What can we do if the API keys can't be made secret?

Use access control in such a way that even with the keys, no harm can be done

1. Limit access to tables
2. Limit access to individual objects
3. Restrict class creation

Let us take each of these in turn.

1. Limiting Access to Tables

For every table you can choose who has access to it and what privileges they have

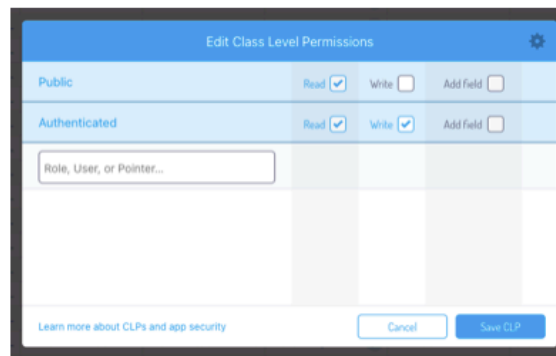
Who has access can be specified with multiple levels of granularity

- Public (anyone, even unauthenticated)
- Authenticated users (requiresAuthentication)
- Specific users
- Roles

Privileges for the entire class

- Get - retrieve individual objects by ID
- Find - query for objects
- Create - create new objects
- Update - modify existing objects
- Delete - delete objects
- Add Fields - add new fields to the schema

The image below shows the Parse UI for setting Class-level permissions



Practical Implication: For your applications, you can prevent non-authenticated users to access your tables

2. Object-Level Permissions

Even if now you only allow logged in users, it would still not be desirable that an unfriendly user creates an account and then

- reads other users data
- or even starts deleting other people's data!

This is where the **Access Control Lists** concept come into play. They allow you to set **fine-grained permissions** for every row in your table. Usually they are **created at the same time** as the object.

Who can ACL permissions apply to?

- Public
- Specific users
- Roles

Privileges per object

- Read - can retrieve/query this object
- Write - can update or delete this object

Examples

User creates a private note

In the following example, a logged in user, creates a private note and ensures that it is only himself that can access that note:

```
const Counter = Parse.Object.extend("Counter");
const privateCounter = new Counter();
privateCounter.set("name", "Times Checked Twitter");
privateCounter.set("count", 42);
privateCounter.setACL(
  new Parse.ACL(Parse.User.current()));
privateCounter.save();
```

Read for public but write only for owner

It is sometimes desirable that an object can be **read by other users**, but just **can not be written by them**. For such a case the `Parse.ACL` object offers the `setPublicReadAccess(true)` method call:

```
const Post = Parse.Object.extend("Post");
const publicPost = new Post();
publicPost.set("content", "I love technical interaction design");
publicPost.setACL(new Parse.ACL(Parse.User.current()));
publicPost.setPublicReadAccess(true);
publicPost.save();
```

Access control lists can be modified every time an object is saved.

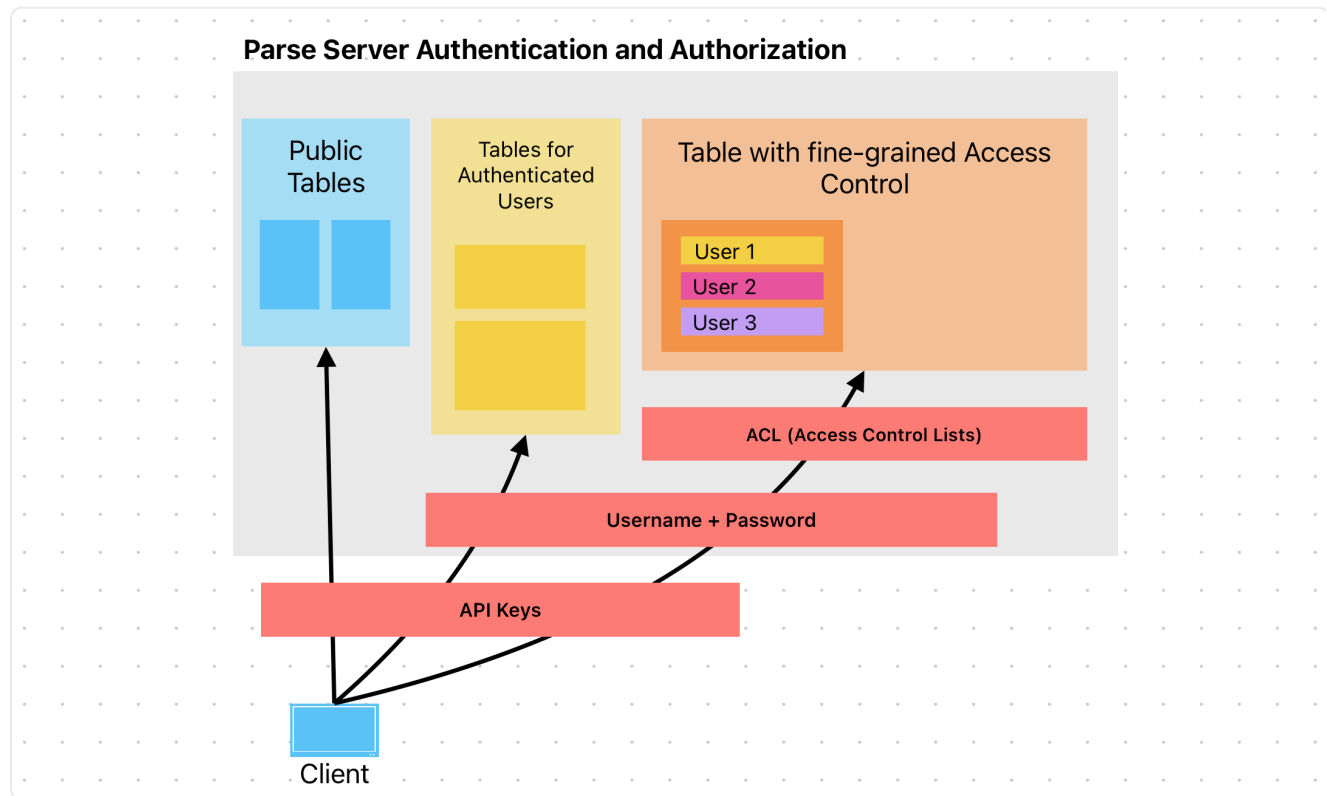
3. Restricting Class Creation

Surely, not all users should be allowed to create classes either!

Under `App Settings > Server Settings > Client Class Creation` you can specify if your expect users to be allowed to create new classes in your database. Probably you do not want that.

4. Combining Authorization Methods to Harden the Security of an Application

The methods above should be combined together to strengthen the DB access for your application.



In practice, there's no real reason to have any public tables. If it's a public list of objects, they can be hardcoded in the application.

Case Study: ToDo26

The model we are hardening

```
erDiagram
    _User ||--o{ TodoItem : owns
    List ||--o{ TodoItem : contains
    _User {
        objectId string PK
        username string
        email string
    }
    List {
        objectId string PK
        name string
    }
    TodoItem {
        objectId string PK
        text string
        done boolean
        list pointer FK
        owner pointer FK
    }
```

Two pointers. Parse adds `objectId`, `createdAt` and `updatedAt` to every class for free, so the only fields we declare are the ones that mean something. The `owner` pointer is the one that arrived today, and everything that follows hangs off it.

The query that looks like security, and is not

`fetchTodos` from last week lives in `src/services/todoService.js`. Now that a to-do has an owner, it grows one more constraint:

```
export async function fetchTodosByList(list) {
  const query = new Parse.Query(TodoItem);
  query.equalTo("list", list);
  query.equalTo("owner", Parse.User.current());
  query.ascending("createdAt"); // oldest first

  const results = await query.find();
  return results.map(toPlainObject);
}
```

- Note the multiple query conditions
- Note the ordering constraint

But there's a problem: the query filter is a convenience for *our* client. It is not a rule on the server, and nothing obliges anyone to send it.

And they do not need to tamper with your client to leave it out. They do not need your client at all: `npm install parse`, twenty lines of Node, and the App ID and JavaScript key they read out of the

bundle you shipped them. Then they run whatever query they like, against the same server, with the same keys — and today it answers.

Access Controls when creating a new Todo item

```
export const createTodoItem = async (text, list) => {

  const currentUser = Parse.User.current();

  const item = new TodoItem();
  item.set("text", text);
  item.set("list", list);
  item.set("done", false);
  item.set("owner", currentUser); // ← Links to user!

  // Set ACL so only creator can read and write
  const acl = new Parse.ACL();
  acl.setReadAccess(currentUser, true);
  acl.setWriteAccess(currentUser, true);
  item.setACL(acl);

  try {
    const result = await item.save();

    // the same plain-object conversion as everywhere else
    return toPlainObject(result);
  } catch (error) {
    console.error("Error creating todo:", error);
    throw error;
  }
};
```

Observation: A more concise way

```
const acl = new Parse.ACL();
acl.setReadAccess(currentUser, true);
acl.setWriteAccess(currentUser, true);
item.setACL(acl);
```

// is equivalent to

```
const acl = new Parse.ACL(currentUser);
item.setACL(acl);
```

Other object-level configurations

Sharing with another user

```
const acl = new Parse.ACL(currentUser); // owner has full access
acl.setReadAccess(otherUser, true);    // other user can read
acl.setWriteAccess(otherUser, true);    // other user can write
```

Public read - owner write

```
const acl = new Parse.ACL(currentUser); // owner has full access
acl.setPublicReadAccess(true); // anyone can read
```

Role-Based Access

```
const acl = new Parse.ACL(currentUser);
acl.setRoleReadAccess("TeamMembers", true);
acl.setRoleWriteAccess("TeamMembers", true);
```

Two types of roles

Application-level roles (Moderators, Admins, Premium Users)

- Created manually in Parse Dashboard or via Cloud Code
- Managed by the app administrators, not end users
- Public read of the role is normal - users should see who the moderators are

User-created roles (e.g. Family, MyTeam, ProjectX)

- Created programmatically by regular users from the app
- Each user manages their own teams
- Private - no reason for others to see them

```
// Say user creates a "TeamMembers" role

const roleACL = new Parse.ACL(currentUser);

const role = new Parse.Role("TeamMembers", roleACL);

// Later our user can add other users to this role
role.getUsers().add(user1);

await role.save();
```

ACL does not work at the field level

In Parse, **ACLs work at the object level, not at the field/column level**. This means you cannot make some fields public and other fields private within the same object. **Example scenario** - You want everyone to **see your todo task name** and done status (public read) - But you want to **keep your time tracking data private** (only you can read)

Problem: You can not set some fields private and some public

```
// This does NOT work - you can't set different permissions per field
const todo = new TodoItem();
todo.set("text", "Write report");           // want this PUBLIC
todo.set("done", false);                    // want this PUBLIC
todo.set("totalTime", 3600000);              // want this PRIVATE
todo.set("currentSessionStart", new Date()); // want this PRIVATE

const acl = new Parse.ACL(currentUser);
acl.setPublicReadAccess(true); // This makes ALL fields public!
todo.setACL(acl);
```

Solution: Split data into two tables with a 1-to-1 relationships

Table 1: TodoItem (Public)

```

const TodoItem = Parse.Object.extend("TodoItem");
const todo = new TodoItem();
todo.set("text", "Write report");
todo.set("done", false);
todo.set("owner", currentUser);

// Public read, owner write
const acl = new Parse.ACL(currentUser);
acl.setPublicReadAccess(true);
todo.setACL(acl);
await todo.save();

```

Table 2: TodoTimeTracking (Private)

```

const TodoTimeTracking = Parse.Object.extend("TodoTimeTracking");

const timeTracking = new TodoTimeTracking();

timeTracking.set("todoId", todo); // Pointer to the TodoItem
timeTracking.set("totalTime", 3600000);
timeTracking.set("currentSessionStart", new Date());
timeTracking.set("owner", currentUser);

// Private - only owner can read and write
const acl = new Parse.ACL(currentUser);
timeTracking.setACL(acl);
await timeTracking.save();

```

Folder Structure

Separate Pages and Reusable Components

My favorite way of organizing

- **pages** – One file per page/view
- **components** – Reusable UI components
- **services** – API/backend logic
- **utilities** – a catch all for things that we don't know where to put yet

```
src/  
├─ assets/  
├─ pages/  
│   ├─ LoginPage.jsx  
│   └─ HomePage.jsx  
├─ components/  
│   ├─ TodoList/  
│   │   ├─ TodoList.jsx  
│   │   └─ TodoItem.jsx  
├─ services/  
│   ├─ authService.js  
│   └─ todoService.js  
├─ constants/  
├─ utilities/  
└─ App.jsx
```

Also, refactor, refactor, refactor. When you find a better organization, go with that.

Reading

From the ParsePlatform.org Guide:

- [Class-level Permissions](#)
- [Object-level Access Control](#)

Exam Questions

1. Why can't Parse API keys be kept secret in a web application?
2. What is the difference between authentication and authorization?
3. What are the three methods of access control in Parse?
4. What is an ACL and at what level does it operate?
5. Explain what this code does:

```
const privateNote = new Note();  
privateNote.set("content", "My secret");  
privateNote.setACL(new Parse.ACL(Parse.User.current()));  
await privateNote.save();
```

6. How would you make an object publicly readable but only writable by the owner?

7. What's the problem with this code if we only validate the length on the client?

```
// Client-side validation  
if (text.length > 200) {  
  alert("Too long!");  
  return;  
}  
await todoItem.save();
```

8. This query returns only the current user's to-dos. Explain why it is nevertheless not a security measure, and what is.

```
const query = new Parse.Query(TodoItem);  
query.equalTo("owner", Parse.User.current());  
const results = await query.find();
```

9. What are the two types of roles in Parse and how do they differ?

10. Why can't ACLs be set at the field level, and what's the workaround?

Something unclear in this section? Open an issue.

Web Service APIs

Source: [Lectures/Technical/Backend/Web-Service-APIs.md](#) — something unclear? [Open an issue](#), or send a pull request.

What is an API?

- *Application Programming Interface* – a generic term referring to an *interface to a system*
- the interface is usually made up of
 - a collection of functions
 - conventions of usage for those functions
- the **system** about whose interface we talk about can be
 - a class (e.g. the [Array API](#) in Javascript)
 - a package (e.g. [Java Concurrency API](#))
 - an application (e.g., the [web browser API](#) implemented by modern browsers)
 - a web service: (e.g. [Spotify API](#), [Twitter API](#))

Focus: Web Service APIs

Interfaces for interacting with **remote servers** by using **web technologies** (HTTP)

Their goal is to enable a **client** to exchange data with a **remote server**

The **client** can be

- a web application running in the browser (on your laptop or phone)
- a native application (e.g. the Spotify app)

The **server** can be:

- your own server (e.g. `api.zeeguu.org` for `zeeguu.org`)
- A 3rd party API (e.g. Spotify, Twitter, OMDb, Google Translate)

The server exposes API **endpoints** following the following conventions

- communication is done via HTTP
- data is usually returned in JSON format
- the version of a service is usually encoded in the URL

e.g. Spotify Endpoints

<https://api.spotify.com/v1/me/player/currently-playing>

METHOD	ENDPOINT	USAGE	RETURNS
POST	/v1/me/player/queue	Add an Item to the User's Playback Queue	-
GET	/v1/me/player/devices	Get a User's Available Devices	devices
GET	/v1/me/player	Get Information About The User's Current Playback	currently playing context
GET	/v1/me/player/recently-played	Get Current User's Recently Played Tracks	play history object
GET	/v1/me/player/currently-playing	Get the User's Currently Playing Track	currently playing object
PUT	/v1/me/player/pause	Pause a User's Playback	-
PUT	/v1/me/player/seek	Seek To Position In Currently Playing Track	-
PUT	/v1/me/player/repeat	Set Repeat Mode On User's Playback	-
PUT	/v1/me/player/volume	Set Volume For User's Playback	-
POST	/v1/me/player/next	Skip User's Playback To Next Track	-
POST	/v1/me/player/previous	Skip User's Playback To Previous Track	-
PUT	/v1/me/player/play	Start/Resume a User's Playback	-
PUT	/v1/me/player/shuffle	Toggle Shuffle For User's Playback	-
PUT	/v1/me/player	Transfer a User's Playback	-

REST APIs

A special case of Web Server APIs that follow a strict convention for managing resources via CRUD operations (create, read, update, delete).

Proposed by Roy Fielding in his 2000 PhD thesis based on the observation that HTTP verbs can be mapped on CRUD actions

Conventions

- text based protocol (JSON, XML)
- strict endpoint naming, see example below
- HTTP verbs are mapped on CRUD actions

CRUD	HTTP	REST
Create	POST	/api/movie
Read	GET	/api/movie/{id}
Update	PUT	/api/movie
Delete	DELETE	/api/movie/{id}

Parse offers as an alternative to the JavaScript API that you've been using until now also a REST API. In a system that manages objects of type `Translation`, the way to get a list of translations is done from the command line as in the following:

```
curl -X GET \
  -H "X-Parse-Application-Id: A17ZSSh904iWQjRfzaz9A0EzFS7YSFLE6xdbv0gm" \
  -H "X-Parse-REST-API-Key: f6GeVUvMTJfkrdVZ0Vkh1lWbwQ0Z9kpV7CEJdPRg" \
  https://parseapi.back4app.com/classes/Counter
```

To understand the command

- `curl` - linux based terminal-based tool used for sending HTTP requests
- `-X` specifies the HTTP verb to be used by `curl`
- `-H` specifies headers (in our case, we send two headers that identify us)

A REST API can be called from the command line as above, but normally it is called from within the programming language. Below you have an example of creating a new counter object and uploading it to the above application. Since we are creating a new object, we use the `POST` HTTP verb:

```
const postData = {
  name: "biking distance (in km)",
  to: "10",
};

try {

  const response = await fetch("https://parseapi.back4app.com/classes/Counter/", {
    method: "POST",
    headers: {
      "X-Parse-Application-Id": "AL7ZSSh904iWQjRfzaz9A0EzFS7YSFLE6xdbv0gm",
      "X-Parse-REST-API-Key": "f6GeVUVMTJfkrdVZ0Vkh1lWbwQ0Z9kpV7CEJdPRg",
    },

    body: JSON.stringify(postData),

  });

  if (!response.ok) {
    const message = "Error with Status Code: " + response.status;
    throw new Error(message);
  }

  const data = await response.json();
  console.log(data);

} catch (error) {
  console.log("Error: " + error);
}
```

TODO in class:

- Run this code with node

- Run a GET request

Relationship between the `Parse` object and the Parse REST API

- The `Parse.Object` and `Parse.Query` objects are built *on top of* REST calls
- They expose to you the programmer a nicer and simpler interface
- You could always simply use the REST API with `fetch` w/o the `Parse.Object` and `Parse.Query`

Two Challenges When Designing Web Service APIs

The main two challenges are:

1. **Authentication** - How do you ensure that only the callers you want use your endpoints?
2. **Authorization** - How do you ensure that a user does not overreach?

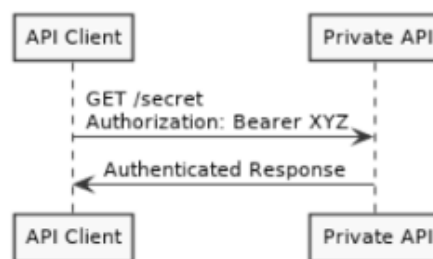
Authentication

API keys

Most popular solution for authentication when your application needs to be using a third-party API (e.g. Google Translate, Spotify API, Twitter API, etc.)

How does it work?

- API Key = Unique generated value is assigned to a user
- Key is sent with every request
- Authentication key is to be kept secret



Why?

- Faster than always sending username / password
- Easy to revoke in case user access should be terminated

Example: If you go to [omdbapi.com](https://www.omdbapi.com/) and get a key, then you can get information about a given movie as below:

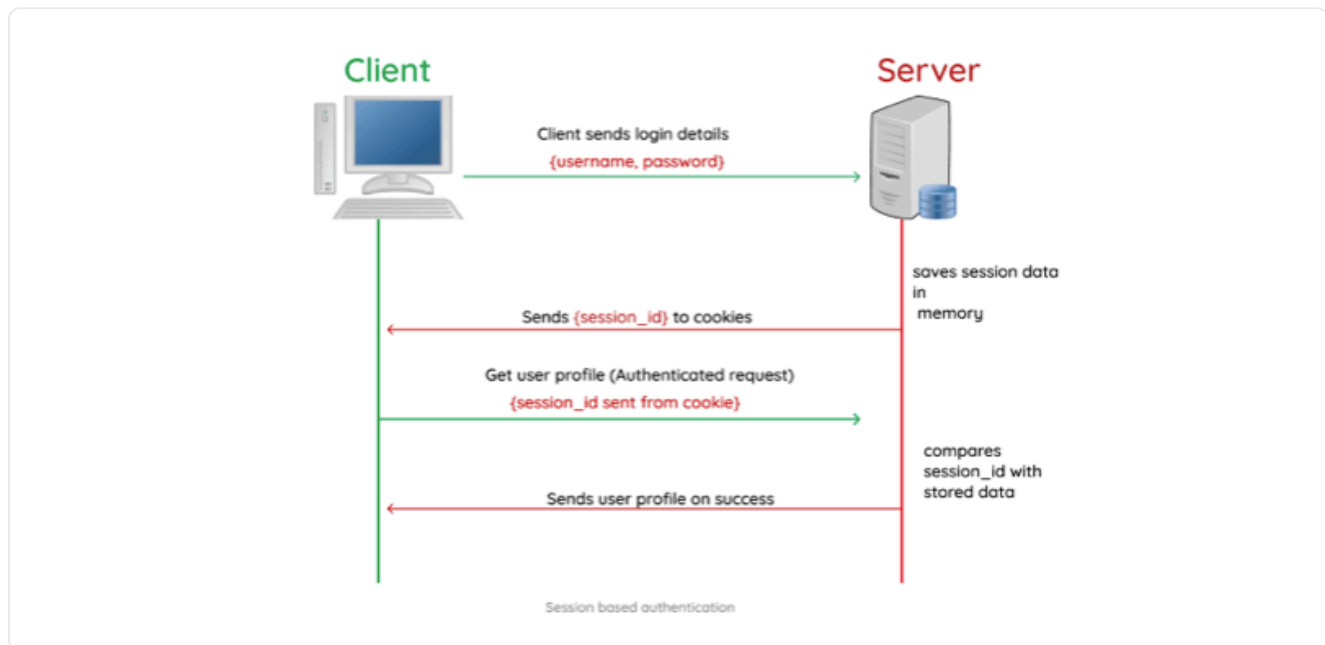
```
fetch("https://www.omdbapi.com/?t=guardians%20of%20the%20galaxy&apikey=955936f0")
  .then(response => response.json())
  .then(data => console.log(data));
```

Session Based Authentication

Most popular solution for authenticating users in a client-server architecture

Addresses the limitations that come with the stateless nature of HTTPS

Session secret key is generated after login on the server, stored in the cookie, and from then on sent always back to the server with every request.



For your projects

- harden security of your app by adding access control

Something unclear in this section? Open an issue.

Not discussed in the course

Something unclear in this section? Open an issue.

Relationships Between Object Classes

Source: [Lectures/Technical/Backend/Relationships-Between-Object-Classes.md](#) — something unclear? [Open an issue](#), or send a pull request.

So far there is one class, and no modeling was required: a to-do has a text and a done flag, and that is the whole design. This note is about what you have to decide the moment there is a second kind of thing in the application — lists that to-dos belong to, users who own them — because how two classes refer to each other is the decision that is hardest to undo once there is data in the database. It assumes you have already written [the four operations against one class](#).

You need to think ahead about the database model as soon as there is more than one kind of thing in your application. The main questions are:

1. What are the types of objects in my domain model?
2. What are the relationships between them?

Todos belong to lists

A single flat pile of to-dos stops being useful somewhere around thirty items. What people actually want is *Personal*, *Apartment*, *Bachelor project* — several lists, each with a name. So:

```
erDiagram
    List ||--o{ TodoItem : contains
    List {
        objectId string PK
        name string
    }
    TodoItem {
        objectId string PK
        text string
        done boolean
        list pointer FK
    }
```

Note what we did *not* do: we did not add a `list` string field to `TodoItem`.

A string would be enough to group them on screen today, and it would be inefficient the moment anyone wants to rename a list.

This is called `normalization` in databases.

If a concept is expressed in a single place, it's easy to change. In our case, renaming a list: we rename it in a single place.

Benefit of a class/table over an attribute is that the class/table can be enriched with more properties later

Our lists could get new properties: priority, etc. Moreover, next week we will want to share a list with other users. If a list is a first class entity in the DB that becomes easily possible.

Note: The table name in the DB does not have to match the component in the react app

And note the three names now in play, each in its own layer: `List` is a class in the database, `ToDoList` is the React component that draws one.

Something unclear in this section? [Open an issue.](#)

The same idea has three names, depending on the context

In a relational database	In Parse	In OO lingo
table	class	class
row	object	object
column	field	attribute
foreign key	pointer	a reference
join table	a class with two pointers	—

So a pointer is Parse's foreign key. It holds which object in which class, and nothing else.

Something unclear in this section? [Open an issue.](#)

One-to-many relationships are done with pointers

Set a pointer by handing `set()` the whole object, not its id:

```
const List = Parse.Object.extend("List");
const TodoItem = Parse.Object.extend("TodoItem");

const item = new TodoItem();
item.set("text", "Call the landlord");
item.set("done", false);
item.set("list", apartmentList); // ← the object itself, not its id
await item.save();
```

Now we can query all the to-dos in a list:

```
const query = new Parse.Query(TodoItem);
query.equalTo("list", apartmentList);
const results = await query.find();
```

or get the list a to-do belongs to:

```
const list = item.get("list");
```

with one catch that will bite you: what comes back from `get("list")` is a Parse object that knows its `id` but has **not** fetched its fields. Asking it for `get("name")` gives you `undefined`. Either fetch it, or — much better — tell the query to bring the lists along:

```
const query = new Parse.Query(TodoItem);
query.include("list"); // ← fetch the pointed-to objects too
const results = await query.find();
results[0].get("list").get("name"); // now this works
```

One request instead of one-per-to-do. We will have more to say about this in the lecture on efficient communication with the backend.

Something unclear in this section? [Open an issue](#).

Many-to-many relationships are done with a join table

Say a to-do can be tagged with several labels, and a label applies to many to-dos. Create a class whose job is to represent one pairing:

```
const TodoLabel = Parse.Object.extend("TodoLabel");

const todoLabel = new TodoLabel();
todoLabel.set("todo", todoItem);
todoLabel.set("label", label);
await todoLabel.save();
```

Two pointers, one per side. That is all a join table is.

Why a join table rather than anything cleverer?

Because the relationship itself will eventually want to carry information, and only a class can hold information:

```
todoLabel.set("addedBy", someUser);
todoLabel.set("addedAt", new Date());
todoLabel.set("order", 1);
```

The moment you need *when* the label was added, or *who* added it, or in *what order*, a join table already has room for it and the alternatives do not.

Something unclear in this section? [Open an issue.](#)

Note: do not model relationships with `Parse.Relation`

You will meet `Parse.Relation` in the documentation, which is Parse's built-in way of doing many-to-many. It is less typing and it cannot carry any information about the relationship, so we are not going to use it. Knowing that it exists is enough.

Something unclear in this section? [Open an issue.](#)

Note: do not model relationships with arrays

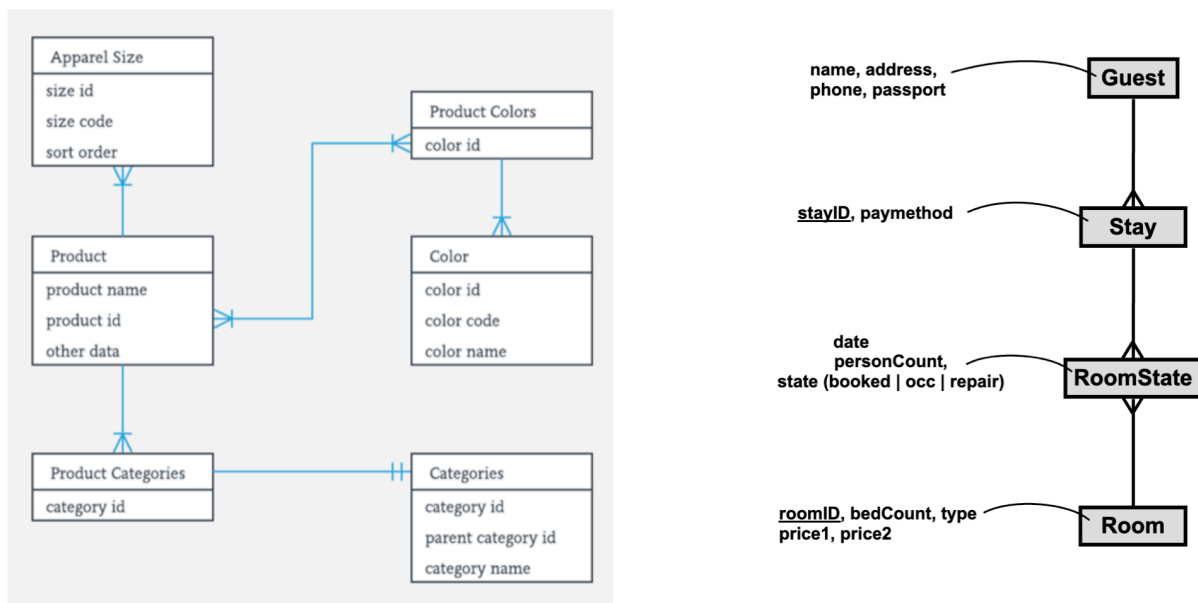
Parse lets you store an array of objects in a field, and it is tempting for small collections. Resist it: an array has no room for information about the relationship, it has to be rewritten in full to add one element — the whole-array problem from the beginning of this lecture, all over again — and it gets slow and awkward as soon as it is not tiny. Pointers for one-to-many, a join table for many-to-many. Those two cover everything you need this semester.

Something unclear in this section? Open an issue.

The notation matters less than being able to explain your model

Use whichever notation you prefer. Two that I like are:

1. On the left hand side is the most popular way of showing attributes
 - crow's feet show cardinality
 - attributes are listed in the box
2. On the right hand side is a compressed approach proposed by Søren Lauesen, ex-professor at ITU



No matter which notation you use, the most important aspect is being able to communicate the way all the relevant data for your application domain is saved in the database.

Something unclear in this section? Open an issue.

Exam Questions

1. What are the two questions you have to answer about your domain before you create any classes?
2. Your app has lists and to-dos. Which class gets the pointer, and why not the other one?
3. Why is a join table preferred over an array field for a many-to-many relationship?
4. Why model a list as its own class rather than as a string field on each to-do?

References

The documentation on ParsePlatform.org

- [Relationships](#) - this is very good and must be read attentively – it will really help with modeling